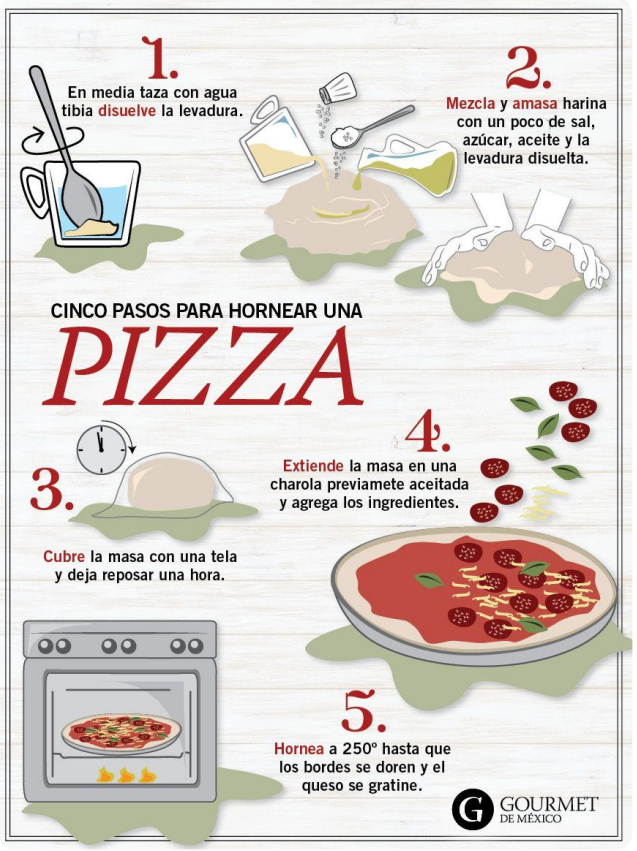
Eficiencia de los algoritmos

Cómo analizar tiempo y espacio para tomar mejores decisiones de diseño de software

¿Por qué medir eficiencia?

"Un algoritmo es como una receta; medir eficiencia es evaluar cuánto tarda y cuántos recursos usa".



¿Qué es la Eficiencia Algorítmica?

La eficiencia algorítmica es una manera estandarizada y matemática de responder:

“¿Qué tan rápido y que tantos recursos necesita este algoritmo cuando el problema se hace grande?”

Salida

T(n)

S(n)

¿Por qué medir la Eficiencia? Importancia Práctica

Escalabilidad en Sistemas Reales

- Crucial para manejar millones de usuarios o peticiones simultáneas sin colapsar la infraestructura.

Costos y Sostenibilidad

- Algoritmos eficientes reducen el consumo de CPU/memoria, bajando la factura de la nube y la huella de carbono.

Experiencia de Usuario (UX)

- La latencia baja es vital: los usuarios abandonan aplicaciones que tardan más de unos segundos en responder.

Selección de Estructuras de Datos

- Permite elegir informadamente entre Array, LinkedList, HashMap o Tree según el caso de uso.

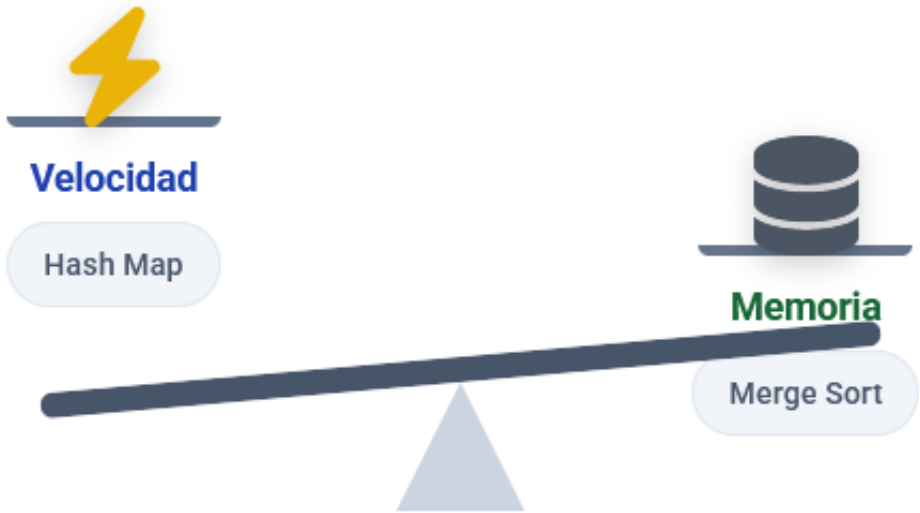
Recursos Computacionales: Tiempo vs Espacio

Tiempo de Ejecución

Número de **operaciones elementales** realizadas en función de n. Incluye comparaciones, asignaciones y accesos a memoria.

Espacio en Memoria

Memoria **adicional** requerida más allá de la entrada. Incluye espacio en pila (recursión), heap y estructuras auxiliares.



Ejemplos de compromiso Tiempo-Espacio:

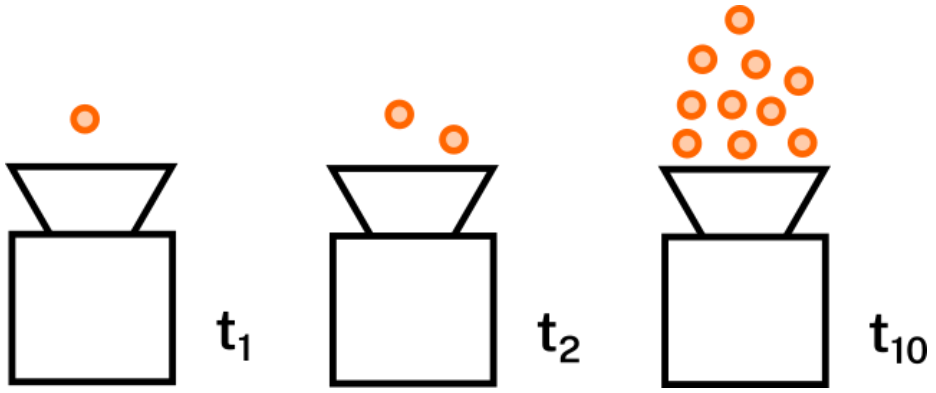
Caching	Compresión
Usa memoria para ganar tiempo	Usa tiempo (CPU) para ganar espacio

Complejidad temporal

- Mide el número de operaciones que un algoritmo necesita para ejecutarse en función del tamaño de la entrada.

Idea clave: no se trata del tiempo real en segundos, sino de cómo crece el número de pasos cuando aumentan los datos.

Analogía: Es como medir cuánto tiempo tarda una receta según el número de comensales. Si duplicas los invitados, ¿se duplica el tiempo de preparación o crece más rápido?



Complejidad espacial

- Mide la cantidad de memoria que un algoritmo necesita para ejecutarse.
- Incluye:
  - Variables simples.
  - Estructuras de datos auxiliares (arreglos, listas, pilas).
  - Espacio para la recursividad (pila de llamadas).

Analogía: es como medir cuántos utensilios o ingredientes adicionales requiere una receta. No solo importa el tiempo, también el espacio en la cocina.

Comparación
Complejidad temporal vs Complejidad espacial

Ambos conceptos son complementarios: un algoritmo puede ser rápido, pero consumir mucha memoria, o usar poca memoria, pero ser lento.

En la practica, se busca un balance entre tiempo y espacio según el problema y los recursos disponibles

Notación Asintótica: Visión General

- La notación asintótica es una herramienta matemática que usamos para describir cómo crece el tiempo o el espacio que necesita un algoritmo cuando aumenta el tamaño de la entrada. Lo importante es que no mide segundos ni bytes exactos, sino la tendencia de crecimiento.

🌱 Idea básica

- Imagina que tienes un algoritmo que tarda más conforme aumentan los datos.
- La notación asintótica nos dice qué tan rápido crece ese tiempo en relación con el tamaño de la entrada .
- Se enfoca en el comportamiento cuando es muy grande (por eso se llama asintótica).

Tipos principales

O

Big O (O grande) PEOR CASO

Establece un **límite superior**. Garantiza que el algoritmo no tardará más que esto.

$f(n) \leq c \cdot g(n)$

Ω

Big Omega (Ω) MEJOR CASO

Establece un **límite inferior**. El algoritmo tardará al menos esto.

$f(n) \geq c \cdot g(n)$

Θ

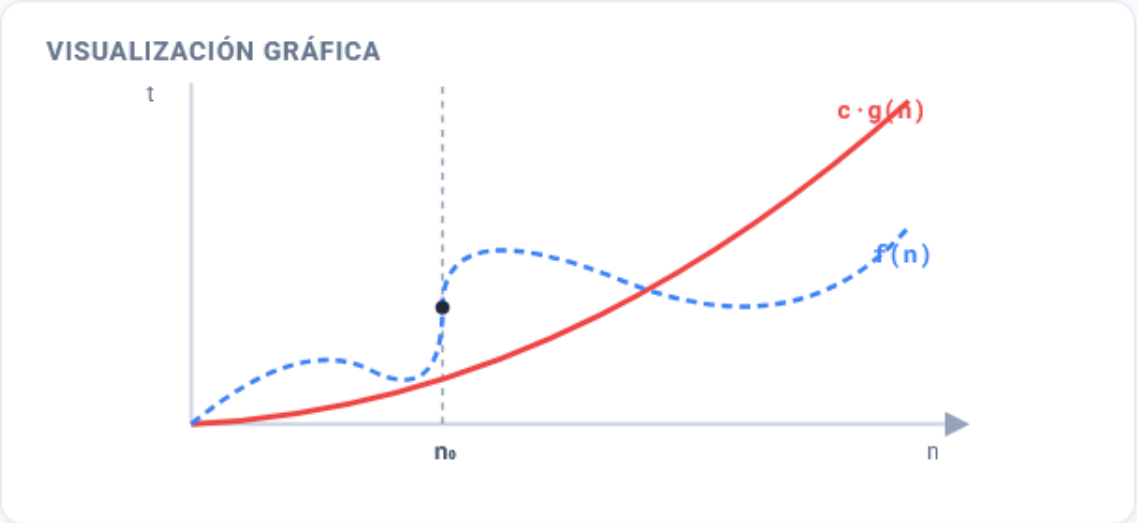
Big Theta (Θ) CASO AJUSTADO

Límite ajustado (superior e inferior). Describe el crecimiento exacto.

$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

Big O (O): Peor Caso

- Representa el **techo de crecimiento**. Nos garantiza que el algoritmo no consumirá más recursos que los indicados por la función g(n), salvo por factores constantes.



¿Por qué es importante?

Es la métrica de seguridad. En sistemas críticos (frenos de auto, equipos médicos), debemos conocer el **peor escenario posible** para garantizar que el sistema siempre responda a tiempo.

Big Omega (Ω): Mejor Caso

La notación Big Omega (Ω) describe una **cota inferior asintótica**. Nos dice que el algoritmo tomará *al menos* esta cantidad de recursos para tamaños de entrada grandes.

Ejemplos Clásicos

Búsqueda Lineal (Encontrar el primero)

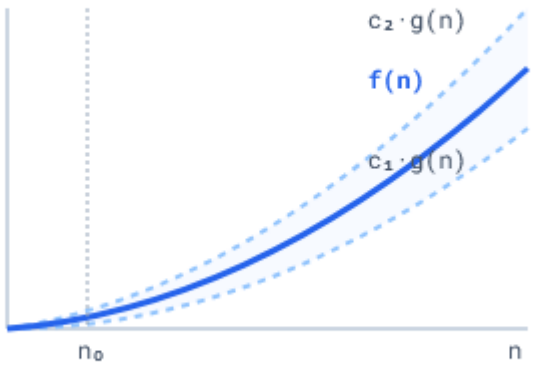
Si el elemento buscado está en la primera posición del arreglo.

`Ω(1)`

Big Theta (Θ): Caso Ajustado / Promedio

- Decimos que **f(n) ∈ Θ(g(n))** si la función está acotada tanto superior como inferiormente por la misma función g(n) (multiplicada por constantes).
- **Intuición:** La función crece exactamente a la misma tasa que g(n). Es una "cota ajustada". Esto ocurre cuando el límite superior (O) y el límite inferior (Ω) son del mismo orden.

VISUALIZACIÓN DEL "SANDWICH"



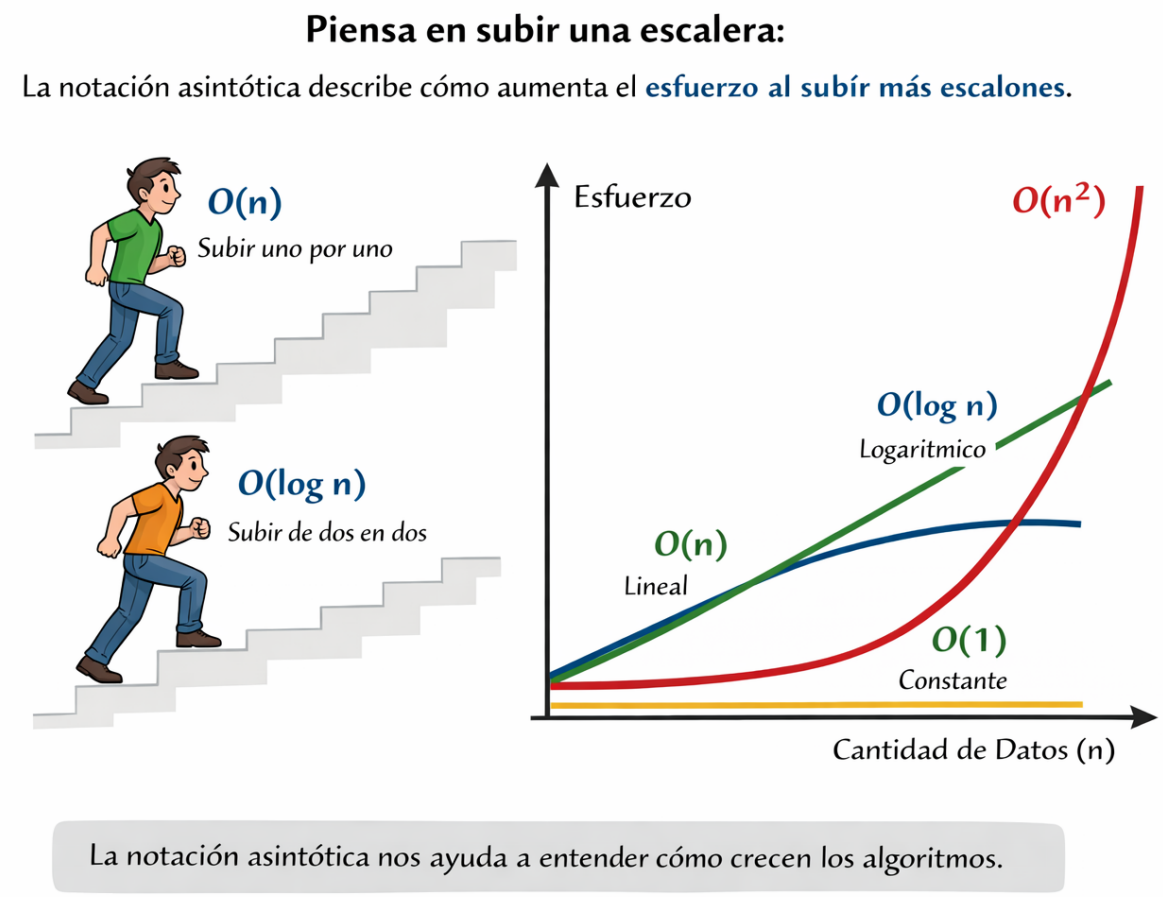
Búsqueda Lineal `Θ(n)`

En promedio, encontramos el elemento a la mitad del arreglo (n/2 pasos). Al ignorar la constante 1/2, el crecimiento es lineal.

Ejemplos Sencillos

Piensa en subir una escalera:

- Si cada sección representa un dato, la notación asintótica describe como aumenta el esfuerzo al subir más escalones.
- Subir uno por uno -- esfuerzo lineal O(n)
- Subir de dos en dos – esfuerzo más eficiente, parecido a O(log n)



Notación Big-O

Describe cómo crece el tiempo de ejecución cuando aumenta el tamaño de entrada (n).

Complejidad	Nombre	Ejemplo
O(1)	Constante	Acceder a un elemento de una lista
O(n)	Lineal	Recorrer una lista
O(n^2)	Cuadrática	Doble ciclo anidado
O(log n)	Logarítmica	Búsqueda binaria

Ejemplo 1: Complejidad O(n)

Recorremos una lista y sumamos sus elementos.

```python
def suma_lista(numeros):
    total = 0
    for numero in numeros:
        total += numero
    return total
```

Análisis

- Recorre n elementos.
- Complejidad: **O(n)**

¿Qué pasa si la lista tiene 1 millón de elementos?

Ejemplo 2: Complejidad O(n^2)

- Comparar todos los pares posibles.

```python
def comparar_pares(numeros):
    contador = 0
    for i in range(len(numeros)):
        for j in range(len(numeros)):
            contador += 1
    return contador
```

Análisis

- Dos ciclos anidados
- n x n
- Complejidad: **O(n^2)**

¿Cómo afectaría esto en un dataset de imágenes para IA?

Ejemplo 3: O(log n) – Búsqueda Binaria

```python
def busqueda_binaria(lista, objetivo):
    izquierda = 0
    derecha = len(lista) - 1

    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2

        if lista[medio] == objetivo:
            return medio
        elif lista[medio] < objetivo:
            izquierda = medio + 1
        else:
            derecha = medio - 1

    return -1
```

Análisis

- Complejidad: **O(log n)**

¿Por qué es más eficiente que recorrer toda la lista?

¿Cómo se analiza la complejidad de un algoritmo?

Analizar la complejidad de un algoritmo significa responder esta pregunta:

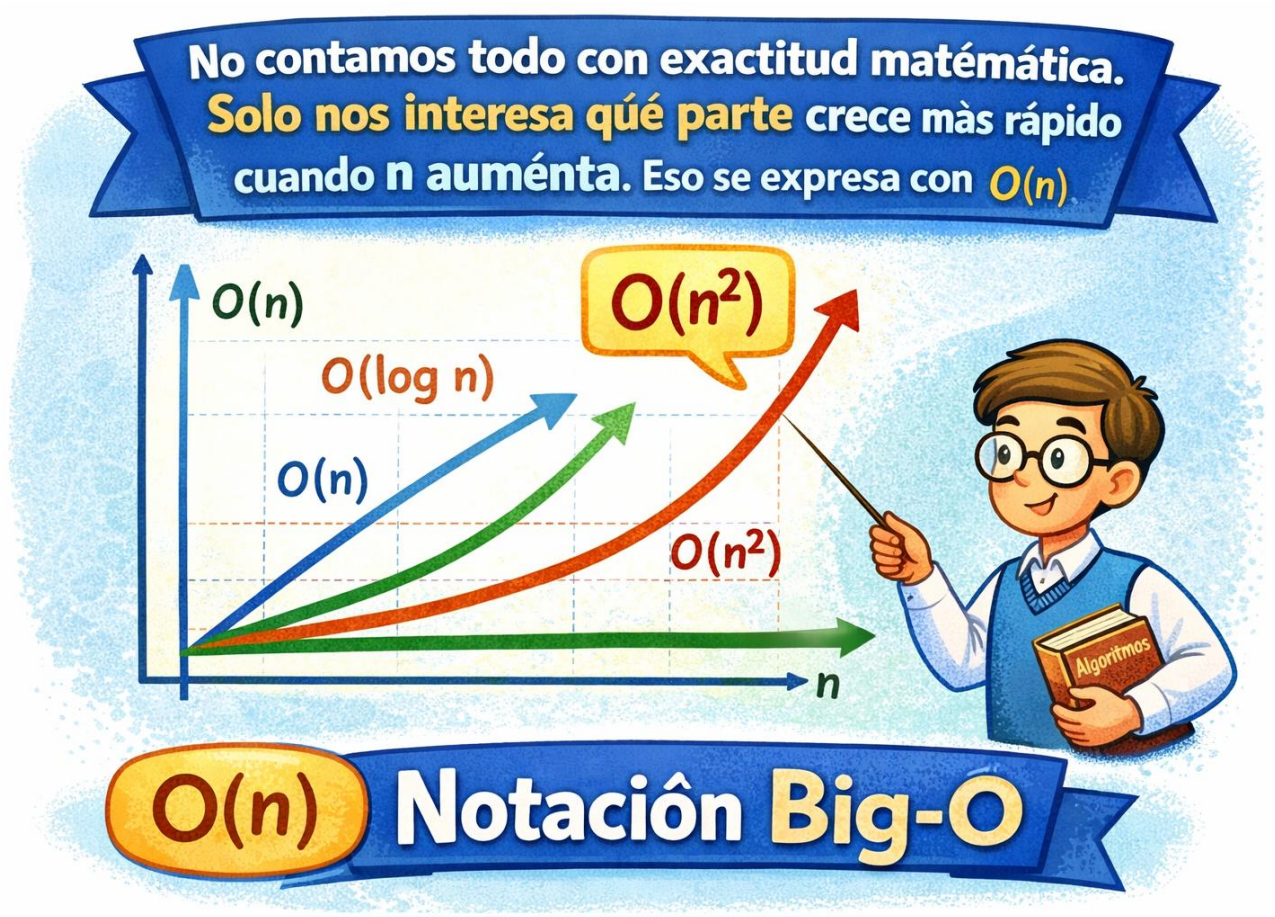
¿Cómo crece el número de operaciones cuando aumenta el tamaño de los datos (n)?

No medimos segundos reales, sino **qué tan rápido crece el trabajo del algoritmo**.

Paso 1: Identifica qué depende de n

- La clave es observar:
  - 🔁 Ciclos (for, while)
  - 🔄 Ciclos anidados
  - 🔎 Operaciones de búsqueda
  - 📦 Uso de estructuras de datos

Paso 2: Cuenta operaciones dominantes



No contamos todo con exactitud matemática. Solo nos interesa **qué parte crece más rápido** cuando n aumenta.

Eso se expresa con **Notación Big-O**.

Reglas Básicas (Muy Simples)

1. Un ciclo que recorre toda la lista → O(n)

```python
def imprimir(lista):
    for elemento in lista:
        print(elemento)
```

¿Qué pasa aquí?

- Si la lista tiene 10 elementos → 10 operaciones
- Si tiene 100 → 100 operaciones
- Si tiene n → n operaciones

Complejidad: **O(n)**

2. Dos ciclos anidados → $O(n^2)$

```python
def pares(lista):
    for i in lista:
        for j in lista:
            print(i, j)
```

Análisis:

- El primer ciclo corre n veces
- El segundo ciclo corre n veces por cada iteración
- Total → n × n

Complejidad: **$O(n^2)$**

3. Operación constante → O(1)

```python
def obtener_primero(lista):
    return lista[0]
```

Siempre hace 1 operación.

Complejidad: **O(1)**

No importa si la lista tiene 10 o 1 millón de elementos.

4. Búsqueda binaria → O(log n)

```python
def busqueda_binaria(lista, objetivo):
    izquierda = 0
    derecha = len(lista) - 1

    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2

        if lista[medio] == objetivo:
            return medio
        elif lista[medio] < objetivo:
            izquierda = medio + 1
        else:
            derecha = medio - 1

    return -1
```

Cada vez divide el problema a la mitad.
Si n = 1,000,000
Solo necesita alrededor de 20 pasos.

Complejidad: **O(log n)**

Método Paso a Paso para Analizar Cualquier Algoritmo

1. Identifica ciclos.
2. ¿Están anidados?
3. ¿Se dividen los datos en cada paso?
4. Elimina constantes.
5. Quédate con el término que crece más rápido.

Ejercicio 1: Determina la complejidad

```
def imprimir_elementos(lista):
    for elemento in lista:
        print(elemento)
```

2. Determina la complejidad

```python
def doble_recorrido(lista):
    for i in lista:
        for j in lista:
            print(i, j)
```

3. Optimiza el algoritmo

Encuentra si un número se repite en la lista.

```
def tiene_duplicados(lista):
    for i in range(len(lista)):
        for j in range(i + 1, len(lista)):
            if lista[i] == lista[j]:
                return True
    return False
```

4. Analiza y mejora

```python
def buscar(lista, objetivo):
    for elemento in lista:
        if elemento == objetivo:
            return True
    return False
```

Practica

Analiza la complejidad:

```python
for i in range(n):
    print(i)
```

Optimiza

```python
def buscar_max(lista):
    maximo = lista[0]
    for i in range(len(lista)):
        for j in range(len(lista)):
            if lista[j] > maximo:
                maximo = lista[j]
    return maximo
```